GitHub Deployment Flow & Rollback Guide

1. Branch Strategy Overview

develop | feature/user-auth (New feature) → commit abc123 (Merged to develop) | **DEV Environment**

↓ deploy

staging | develop → staging (PR merged) → commit abc123 (Testing in stage) | **STAGE Environment**

↓ deploy

main | staging → main (PR merged) → commit abc123 (Live! Tag: v1.2.0) | **PROD Environment**

↓ deploy

✓ **Same commit (abc123) flows through all environments**
This ensures what you test in dev/stage is exactly what goes to production!

2. Complete Deployment Process

1

Create Feature Branch

```
git checkout develop git pull origin develop git checkout -b feature/add-payment # Work on your feature, make commits git commit -m "Add payment integration"
```

2

Deploy to DEV

```
git push origin feature/add-payment # Create PR: feature/add-payment → develop # After review, merge PR # CI/CD auto-deploys to DEV environment
```

Test thoroughly in DEV!

3

Promote to STAGING

```
# Create PR: develop → staging # After approval, merge PR # CI/CD auto-deploys to STAGING environment
```

QA team tests in STAGING

4

Deploy to PRODUCTION

```
# Create PR: staging → main # Require 2+ approvals # After approval, merge PR # CI/CD deploys to PRODUCTION git tag -a v1.2.0 -m "Release 1.2.0" git push origin v1.2.0
```

Monitor production metrics!

3. Rollback Strategies When Things Go Wrong 🆘

Git Revert | Redeploy Previous | Hotfix

Option 1: Git Revert (Recommended) ✅

Best for: Quick rollback while preserving history

```
# 1. Find the bad commit git log --oneline # Output shows: def456 (bad), abc123 (good) # 2. Create revert commit git checkout production git revert def456 # This creates a NEW commit that undoes def456 # 3. Push and deploy git push origin production # CI/CD auto-deploys the reverted state
```

Advantages:

- Preserves complete history (audit trail)
- Safe - doesn't rewrite history
- Can be reverted again if needed
- Other developers won't have conflicts

Important: Also revert in staging and develop to keep branches in sync:

```
git checkout staging git revert def456 git push origin staging git checkout develop git revert def456 git push origin develop
```

4. Rollback Strategy Comparison

✅ Git Revert

Speed: Fast (5-10 min)
Risk: Low
History: Preserved
Use when: Bad code merged, need quick rollback

⚡ Redeploy Previous

Speed: Very fast (2-5 min)
Risk: High if using force push
History: Lost if using reset
Use when: Deployment issue, emergency only

🔧 Hotfix Branch

Speed: Moderate (20-60 min)
Risk: Low
History: Preserved + tested
Use when: Need to fix and test properly

5. Real-World Rollback Scenario

Scenario: Payment feature causes 500 errors in production

1. **Detect Issue (2:30 PM)**
   Monitoring alerts show 500 errors spiking. Users can't checkout!

2. **Identify Cause (2:35 PM)**
   Logs show payment integration bug. Last deploy was commit def456

3. **Execute Rollback (2:40 PM)**

```
git checkout production git revert def456 git push origin
production # CI/CD deploys in 3 minutes
```

4. **Verify Fix (2:45 PM)**
   Errors stop. Checkout working. Total downtime: 15 minutes

5. **Fix Properly (Next Day)**

```
# Create new branch to fix the bug git checkout develop git
checkout -b bugfix/payment-integration # Fix the bug, test
in dev, then staging, then prod
```

6. Best Practices & Tips

Before Deploying:

- **Tag releases:** Always tag production deploys (v1.2.0) so you can easily find them

- **Test thoroughly:** Each environment should catch different types of issues
- **Small changes:** Deploy smaller, more frequent changes (easier to rollback)
- **Feature flags:** Use feature flags to disable new features without rolling back

During a Crisis:

- **Don't panic:** Follow your rollback playbook
- **Communicate:** Alert team immediately in Slack/chat
- **Preserve evidence:** Save logs and errors before rolling back
- **Revert first, debug later:** Get production stable, then investigate

After Rollback:

- **Sync all branches:** Make sure develop, staging, production all match
- **Write postmortem:** What went wrong? How to prevent next time?
- **Fix properly:** Create new branch, fix bug, test through all environments
- **Update monitoring:** Add alerts for this type of issue

```
# Quick reference: Check what's deployed where git log production --
oneline -5 # What's in prod git log staging --oneline -5 # What's in
staging git log develop --oneline -5 # What's in dev # See differences
between environments git log production..staging # What's in staging
but not prod git log staging..develop # What's in dev but not staging
```